

Complete prime-power residue atlases

Every singular branch and exact populations in enormous rectangles

Both smooth chart directions, complete singular lifts, global modular exclusions and kernel-checked nonenumerating two-coordinate counts tied to the original source polynomial.

16

general theorem declarations

7

complete worked atlases

120

independent source census cases

Complete declared prime-power depths and exact modular populations. Global height bounds, general addressing refinement and compiler semantics retain independent obligations.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

Why this is the next highest-value push

The prior release proved complete source-solution covering in a declared box and a single smooth vertical residue class. It deliberately rejected singular vertical derivatives and imposed a 4096-pair coarse-grid budget. Adding only a horizontal chart would repair a useful omission while retaining the larger bottleneck: a local arithmetic engine must preserve every branch, including singular residues that split into many children or disappear at later levels. This push builds that complete multilevel engine and connects its surviving classes to exact nonenumerating two-coordinate populations.

The latest inspected branch includes the concurrent Dresden observatory and formal calendar additions, and the thermal fold census. Those components are preserved. This release concentrates on the arithmetic interface and reuses the existing ResiduePopulation floor-count theorem, rather than duplicating its interval mathematics. The integration is carried by a new general Lean module, an exact sparse-polynomial producer, seven public query operations, seven worked native atlases and a handoff with the next mathematical obligations.

Exact lift alternatives

Let F be the supplied integer polynomial, p a prime and m a positive power of p . Suppose (a,b) is already a root modulo m . Every possible child has coordinates $a+mu$ and $b+mv$, with u and v in the digit interval from zero through $p-1$. Expanding the source gives an exact integer identity, with $c = F(a,b)/m$ and derivative coefficients evaluated at that parent.

$$F(a + mu, b + mv) = m(c + d_x u + d_y v) + m^2 R(u, v)$$

Because p divides m , the general LiftPacket.step theorem proves that divisibility of the child source by mp is equivalent to divisibility of $c+d_x u+d_y v$ by p . This theorem uses the exact identity and elementary integer cancellation. It does not assume an unproved analytic Hensel statement or require one derivative to be invertible.

When the vertical coefficient is nonzero modulo p , the producer solves uniquely for v for every horizontal digit u . When the vertical coefficient is zero but the horizontal coefficient is nonzero, it solves uniquely for u for every v . When both coefficients vanish modulo p , the constant determines the outcome: all p squared digit pairs survive if p divides c , and none survive otherwise. The singular theorem proves this final equivalence; horizontal_unique and vertical_unique prove cancellation in the corresponding smooth charts. The producer checks each resulting child against the original source before advancing.

All parent records include the literal parent, exact constant and derivative values, chosen chart, obstruction flag and complete child list. The final roots and every intermediate level are canonical sorted lists. No preferred ordinate sign, branch or chart is silently

discarded. Work and modulus budgets cause rejection of the whole requested atlas rather than truncation.

Independent completeness and native source binding

The replay implementation does not call lift discovery. For every level it enumerates the entire canonical residue square and evaluates the original polynomial modulo that level's modulus. It reconstructs every parent's children by reducing the independently obtained roots to the preceding modulus. It then checks all chart metadata, exact coefficients, branch lists and the final root set. Canonical JSON comparisons distinguish Boolean coordinates from integer coordinates.

In Lean, rootTable is the canonical residue square filtered by source equality modulo the modulus. AtlasPacket carries positivity, equality with the returned literal roots and a universal source-periodicity theorem. Generated polynomial packets prove the latter directly by modular ring arithmetic. AtlasPacket.complete establishes a biconditional for arbitrary signed integer coordinates: the normalized coordinate pair belongs to the complete table exactly when the original source vanishes modulo the modulus.

Every generated level has its own kernel-checked source root table. Every generated parent has a universal exact expansion identity, a next-level divisibility equivalence and equality of its complete source-defined child table with its returned child list. This checks retained singular branches and branch deaths directly. The exported objects are typed mathematical interfaces; they do not constitute a universal theorem about parsing arbitrary JSON or translating arbitrary Python code into Lean.

Source survival and global obstruction

Any actual integer solution $F(x,y)=0$ belongs to every complete modular atlas, for all signed x and y . No finite box or height bound is needed for this necessary-condition theorem. If a complete atlas is empty, AtlasPacket.empty_obstruction proves that the source has no integer solution anywhere. If it is nonempty, its roots remain necessary local conditions and do not imply that an integer source solution exists.

The worked source $y^2 - x^2 - 2$ illustrates singular obstruction. Its two roots modulo two are both singular. Neither has a child modulo four, because a difference of squares cannot equal two modulo four. The complete final atlas is empty, and the generated native theorem excludes every integer pair. The example is elementary; the reusable capability is preserving and proving the complete branch alternatives for any source within the declared budgets.

Nonenumerating rectangle populations

Each complete canonical root (a,b) defines a product of two arithmetic progressions in a closed rectangle: x has residue a and y has residue b modulo m. Different canonical roots define disjoint cells. The existing one-coordinate count theorem evaluates each axis with floor quotients, and the new `card_cellValues` theorem multiplies the two counts.

`card_candidates` sums those products over the complete finite root table.

$$N_{a,b} = \left(\left\lfloor \frac{x_1 - a}{m} \right\rfloor - \left\lfloor \frac{x_0 - 1 - a}{m} \right\rfloor \right) \left(\left\lfloor \frac{y_1 - b}{m} \right\rfloor - \left\lfloor \frac{y_0 - 1 - b}{m} \right\rfloor \right)$$

The count theorem is an equality with the cardinality of the original-coordinate finite set. Its proof handles negative bounds and empty cells. It does not enumerate the rectangle, assume a density approximation or identify modular candidates with actual equation solutions. Generated huge-box count proofs rewrite cardinality to the short floor sum before evaluating that sum inside the kernel.

For y - x squared, the complete modulo-25 table has 25 roots. In the rectangle where both coordinates range from negative one trillion to positive one trillion, the exact count is 160000000000480000000001. The full box contains 4000000000004000000000001 integer pairs. The large native count is obtained from 25 cells; no candidate list of that size is materialized.

Addressing and complete source scans

The Python population supports direct selection and inverse ranking. Its explicit order is lexicographic in the residue class, followed by ascending x and then ascending y inside that class. Selection subtracts complete cell block sizes and uses quotient and remainder to recover the two progression coordinates. Ranking checks original-coordinate bounds and the source modular condition, locates the unique residue cell and computes the inverse address. This ordering differs from a global lexicographic scan of all points in the rectangle.

The rank and select algorithms are tested against complete signed-box enumerations and round trips at enormous indices. Their general interpreter refinement has not been formalized in this release. The exact count and source-admissibility interfaces have general Lean proofs; these should remain distinct from the tested Python addressing implementation.

A complete source scan first computes the exact surviving population count. If that entire population fits its budget, it scans only those disjoint cells and evaluates F exactly. Oversized populations are rejected before any partial solution list is produced. `solutions_complete` proves that filtering the modular candidates by source equality returns exactly all source solutions in the declared rectangle. Small native worked boxes additionally check equality with their literal returned point lists and expose arbitrary-coordinate completeness theorems.

Worked corpus and independent census

VerticalParabola uses $p=5$ through exponent two and demonstrates a huge nonenumerating count. HorizontalParabola uses $x - y$ squared at $p=2$ through exponent four: every lift uses the horizontal chart because the vertical derivative is always even. SingularCusp uses y squared - x cubed at $p=2$ through exponent three, with complete root counts 2,6,12. SingularCircle uses x squared + y squared and contains singular branch growth and branch death. GlobalObstruction supplies the empty modulo-four difference-of-squares table. Mordell uses y squared - x cubed + 2 at $p=3$ through exponent two, removing an obstructed singular residue while retaining all six modulo-nine roots. Quartic uses y squared - x to the fourth at $p=3$ through exponent two and retains its singular children.

The independent census uses a fixed random seed and 120 sparse polynomial sources. A separate nested Horner evaluator recomputes complete root tables, full signed-box candidate counts and exact source solutions. It also checks rank/select round trips. The census includes eleven empty final tables and thirteen singular parent nodes. Focused tests separately cover source mutations, omitted intermediate children, changed derivatives or chart labels, Boolean coordinates, composite primes, work overflow, modulus overflow, negative coordinates, huge ranks, empty cells and scan rejection without partial results.

Reproduction, budgets and the next handoff

Run `make check-residue-atlas` with the pinned Lean 4.20.0 toolchain and Mathlib dependency. The script regenerates all worked packets, runs focused tests and the independent census, compiles the general module and every native program, and checks theorem dependencies against `propext`, `Classical.choice` and `Quot.sound`. It rejects `sorryAx` and `Lean.ofReduceBool`. Actual execution outcomes belong to the accompanying validation receipt; API outputs continue to mark `execution_verified` as false.

The producer retains the earlier sparse-source limits: at most 24 input terms, degree at most twelve and 256-bit coefficients. Atlas exponents are at most eight; moduli are at most 256; the adjustable lift-work cap is at most two million. Rectangle endpoints have at most 256 bits. Native source emission currently caps the final modulus at 32 and total parent records at 64; these are execution budgets, not mathematical impossibility statements. Default scans admit at most 4096 surviving candidates, with an explicit maximum adjustable scan budget of 100000. Small native point-list checks require a box of at most 1024 pairs and at most 512 modular candidates.

The next highest-value bounded extension is a factored coprime multi-prime product. Prove a coordinate CRT equivalence, combine complete local tables without losing residue combinations, and derive original-coordinate count and addressing semantics for their intersection. Explicit products should be budgeted; large products need a factored representation rather than truncation. Global Mordell height bounds, explicit Matveev premises, integral kernel lattice saturation and a universal source-to-native compiler theorem remain separate obligations.